

AiTSI 2025/26 L - Projekt nr 1 - Testowanie Systemów Informatycznych

Informacje o Projekcie

Skład zespołu

Imię i nazwisko	Rola
Jakub Gerwel	Lider zespołu / Tester
Dominik Gąsowski	Tester / Developer
Arkadiusz Gnidziejko	Tester / Developer
Wojciech Domański	Tester / Developer
Wojciech Gosiewski	Tester / Developer

Tytuł projektu

Smart Expense Buddy – aplikacja do wspólnego rozliczania wydatków grupowych.

Opis projektu

Smart Expense Buddy to aplikacja webowa umożliwiająca grupom użytkowników wspólne zarządzanie wydatkami. Główne funkcjonalności obejmują:

- Rejestracja i logowanie użytkowników (JWT)
- Tworzenie grup rozliczeniowych z rolami (admin / member)
- Dodawanie wydatków z różnymi metodami podziału (równy, kwotowy, procentowy, udziałowy)
- Automatyczne wyliczanie sald i sugerowanie minimalnych przelewów do rozliczenia
- Obsługa wielu walut z kursami pobieranymi z API NBP
- Wydatki cykliczne (dzienne, tygodniowe, miesięczne)
- Zarządzanie kontami osobistymi i transakcjami
- Powiadomienia w czasie rzeczywistym (WebSocket)

Link do repozytorium

<https://github.com/Wojgos/expense-tracker.git>

(repozytorium publiczne)

Stack technologiczny

- **Backend:** Python 3.13, FastAPI, SQLAlchemy (async), PostgreSQL, Redis, APScheduler
 - **Frontend:** React 18 (TypeScript), Vite, Tailwind CSS, React Query
 - **Infrastruktura:** Docker, Docker Compose, Nginx
 - **Testy:** Playwright (Python), pytest, pytest-html
-

Plan Testów (retrospekcyjny)

1. Cele testów

Głównym celem procesu testowania jest weryfikacja, że aplikacja Smart Expense Buddy działa poprawnie, niezawodnie i bezpiecznie. Szczegółowe cele obejmują:

- Sprawdzenie poprawności logiki biznesowej – algorytmy podziału wydatków (equal, exact, percent, shares), obliczanie sald, minimalizacja przelewów
- Weryfikacja bezpieczeństwa – poprawność uwierzytelniania JWT, haszowania haseł, kontroli dostępu (RBAC)
- Potwierdzenie stabilności interfejsu użytkownika – poprawne renderowanie komponentów, obsługa stanów pustych, walidacja formularzy
- Sprawdzenie integracji między frontendem a backendem – poprawność komunikacji API, obsługa błędów sieciowych
- Weryfikacja wydatków cyklicznych – automatyczne generowanie wydatków przez scheduler

2. Zakres testów

Objęte testami:

- Moduł uwierzytelniania (rejestracja, logowanie, JWT, /me)
- Zarządzanie grupami (CRUD, dodawanie/usuwanie członków, role)
- Wydatki grupowe (tworzenie z różnymi split type, pobieranie, usuwanie)
- Rozliczenia (settlement summary, suggested transfers, settle up)
- Konta osobiste i transakcje (tworzenie konta, dodawanie transakcji)
- Wydatki cykliczne (tworzenie, interwały, dezaktywacja)

- Konwersja walut (kurs NBP, przeliczanie)
- Bezpieczeństwo (haszowanie, JWT, kontrola dostępu)
- Interfejs frontendowy (dashboard, strona grup, zakładki, formularze)

Wyłączone z testów:

- Testy wydajnościowe / obciążeniowe
- Testy na urządzeniach mobilnych (responsywność)
- Testy dostępności (accessibility / WCAG)
- Funkcjonalności niezaimplementowane (eksport CSV, filtry zaawansowane, raporty statystyczne)

3. Strategia testowania

Typ testów	Uzasadnienie
Jednostkowe (pytest)	Weryfikacja izolowanej logiki biznesowej – algorytmy split, settlement engine, scheduler, security. Szybkie, niezależne od infrastruktury.
API / integracyjne (Playwright APIRequestContext)	Testowanie endpointów REST bez UI – poprawność statusów HTTP, walidacji, uprawnień. Pozwalają szybko wykryć regresje w backendzie.
E2E frontendowe (Playwright)	Symulacja pełnych ścieżek użytkownika – rejestracja, logowanie, tworzenie grup, dodawanie wydatków. Weryfikują integrację frontend-backend.
Z mockowaniem danych (Playwright page.route())	Testowanie frontendu w izolacji od backendu – szybkie, deterministyczne, pozwalają testować stany brzegowe (puste dane, błędy API, wolne odpowiedzi).
Czarnoskrzynkowe	Przypadki testowe napisane z perspektywy użytkownika, bez znajomości implementacji – weryfikacja wymagań biznesowych.

4. Środowisko testowe

Element	Szczegóły
System operacyjny	Windows 11
Przeglądarka	Chromium (Playwright default)
Rozdzielczość	1280×720 px
Backend	Python 3.13, FastAPI, PostgreSQL 16, Redis
Frontend	Node.js 20+, Vite dev server
Framework testowy	pytest 9.x + pytest-playwright
Raportowanie	pytest-html (raport HTML), JUnit XML

Element	Szczegóły
Konteneryzacja	Docker Compose (PostgreSQL, Redis, backend, frontend)
Baza danych testowa	PostgreSQL w kontenerze Docker (dane tworzone per sesję testową)

5. Harmonogram testów

Okres	Aktywność
Tydzień 1-2 (luty)	Przygotowanie środowiska testowego, konfiguracja Playwright, pierwsze przypadki testowe
Tydzień 3-4 (luty/marzec)	Testy jednostkowe (security, split_calculator, settlement_engine, scheduler)
Tydzień 5-6 (marzec)	Przypadki testowe frontendowe, testy E2E (expenses, groups)
Tydzień 7-8 (marzec/kwiecień)	Testy E2E (accounts/balances, recurring), testy API
Tydzień 9-10 (kwiecień)	Testy z mockowaniem (page.route()), testy storageState
Tydzień 11-12 (kwiecień/maj)	Przegląd wyników, poprawki, dokumentacja, raport końcowy

6. Kryteria wejścia / wyjścia

Kryteria wejścia (warunki przed rozpoczęciem testów):

- Środowisko testowe skonfigurowane i działające (Docker Compose uruchomiony, baza danych dostępna)
- Przypadki testowe przygotowane i zatwierdzone w zespole
- Wymagania biznesowe zdefiniowane (funkcjonalności aplikacji ustalone)
- Brak defektów krytycznych blokujących uruchomienie aplikacji
- Kod źródłowy dostępny w repozytorium Git

Kryteria wyjścia (warunki zakończenia testów):

- Wykonanie minimum 90% zaplanowanych przypadków testowych
- Brak błędów krytycznych (blocker/critical) w głównych ścieżkach użytkownika
- Pokrycie kodu testami jednostkowymi na poziomie minimum 70% dla modułów biznesowych
- Wygenerowanie raportu z wykonania testów (pytest-html)
- Przygotowanie dokumentacji końcowej

7. Ryzyka

Ryzyko	Prawdopodobieństwo	Wpływ	Mitygacja
Niestabilność bazy danych w środowisku Docker	Niskie	Wysoki	Restart kontenerów, dane testowe tworzone per sesję (UUID suffix)
Flaky testy E2E (timeout, asynchroniczność)	Średnie	Średni	Retry logic w fixture'ach, jawne timeouty, <code>wait_for_timeout</code>
Zmiany w API zewnętrznym (NBP)	Niskie	Niski	Mockowanie odpowiedzi NBP w testach, cache w Redis
Brak dostępności członka zespołu	Średnie	Średni	Podział zadań z dokumentacją, testy niezależne od siebie
Problemy z kompatybilnością przeglądarki	Niskie	Niski	Używanie jednej przeglądarki (Chromium) w testach
Konflikty w repozytorium Git	Niskie	Niski	Oddzielne pliki testowe per osoba, code review

8. Role w zespole

Osoba	Rola	Odpowiedzialność
Jakub Gerwel	Lider zespołu / Tester	Koordinacja prac, testy E2E (recurring), testy API (4 testy), testy mockowane (3 testy), storageState (1 test), przypadki testowe (Recurring), testy jednostkowe (scheduler)
Dominik Gąsowski	Tester automatyczny	Testy E2E (dg/), testy API (3 testy), testy mockowane (3 testy), storageState (1 test), testy jednostkowe (security, user)
Arkadiusz Gnidziejko	Tester automatyczny	Testy E2E (groups), testy API (3 testy), testy mockowane (3 testy), storageState (1 test), przypadki testowe (Other), testy jednostkowe (group_crud, schemas)
Wojciech Domański	Tester automatyczny	Testy E2E (expenses), testy API (4 testy), testy mockowane (3 testy), przypadki testowe (Expenses), testy jednostkowe (split_calculator)
Wojciech Gosiewski	Tester automatyczny	Testy E2E (accounts/balances), testy API (4 testy), testy mockowane (3 testy), przypadki testowe (Balances), testy jednostkowe (settlement_engine, expense_schema)

Podsumowanie Przypadków Testowych

Statystyki

Dokument	Liczba przypadków	Autor	Obszar
BALANCES_TEST_CASES.md	10	Wojciech Gosiewski	Konta osobiste, salda, transakcje
EXPENSES_TEST_CASES.md	10	Wojciech Domański	Tworzenie wydatków, walidacja podziału
OTHER_TEST_CASES.md	10	Arkadiusz Gnidziejko	API wydatków, rozliczenia, WebSocket, uprawnienia
RECURRING_EXPENSES_TEST_CASES.md	10	Jakub Gerwel	Wydatki cykliczne, scheduler, interwały
SECURITY_TEST_CASES.md	10	Dominik Gąsowski	Haszowanie haseł, JWT, walidacja tokenów

Łącznie: 50 przypadków testowych (10 na osobę)

Przegląd przypadków testowych wg obszaru

Konta i salda (TC-BAL-001 – TC-BAL-010):

- Wyświetlanie całkowitego salda i sald poszczególnych kont
- Aktualizacja sald po wydatkach i przychodach
- Stan pusty (brak kont)
- Formatowanie walutowe i zaokrąglenia
- Ujemne salda
- Konwersja wielowalutowa
- Tryb prywatności (ukrywanie kwot)
- Usunięcie konta

Wydatki grupowe (TC_001 – TC_010):

- Dodawanie wydatku z podziałem równym (equal)
- Dodawanie wydatku z podziałem kwotowym (exact)
- Dodawanie wydatku z podziałem procentowym (percent)
- Dodawanie wydatku z podziałem udziałowym (shares)
- Walidacja kwoty ujemnej
- Walidacja pustej listy uczestników
- Obsługa daty z przyszłości
- Walidacja pustego opisu
- Walidacja niekompletnych danych exact split
- Kontrola uprawnień (non-member)

Zaawansowane API (TC-NEW-001 – TC-NEW-010):

- Pobieranie szczegółów wydatku po ID
- Walidacja procentowego podziału (suma = 100%)
- Rozliczenie długów (settle up)
- Usunięcie wydatku z kontrolą uprawnień
- Konwersja walut przez API NBP
- Powiadomienia WebSocket w czasie rzeczywistym
- Zarządzanie członkami grupy i RBAC
- Eksport CSV (niezaimplementowane)
- Filtrowanie wydatków (niezaimplementowane)
- Raporty statystyczne (niezaimplementowane)

Wydatki cykliczne (TC-REC-001 – TC-REC-010):

- Automatyczne generowanie wydatku w dniu next_run
- Dezaktywacja zatrzymuje generowanie
- Kontrola dostępu (non-member blokowany)
- Obsługa nieistniejącej grupy
- Pusta kolejka run
- Przesunięcie daty dla interwałów WEEKLY/YEARLY
- 404 dla nieistniejącego recurring ID
- Scheduler z pustą listą członków
- Rollback transakcji przy błędzie schedulera
- Domyślna waluta i mapowanie pól

Bezpieczeństwo (TC-01 – TC-10):

- Haszowanie hasła – maskowanie
- Unikalność soli (salt)
- Weryfikacja poprawnego hasła
- Obsługa UTF-8 i znaków specjalnych
- Odrzucenie błędnego hasła
- Generowanie tokenu JWT
- Poprawność payload (sub)
- Dekodowanie ważnego tokenu
- Obsługa wygasłego tokenu
- Odporność na zniekształcone tokeny

Testy Automatyczne

Testy E2E Frontendowe

Plik	Liczba testów	Autor	Opis
test_accounts_balances.py	8	Wojciech Gosiewski	Konta, salda, transakcje, prywatność
test_expenses.py	8	Wojciech Domański	Dodawanie wydatków (equal, exact, percent, shares, walidacja)
test_groups.py	8	Arkadiusz Gnidziejko	CRUD grup, członkowie, uprawnienia, role
test_recurring_expenses.py	8	Jakub Gerwel	Tworzenie recurring (monthly, weekly), dezaktywacja, walidacja
test_e2e.py (dg/)	8	Dominik Gąsowski	Journey tests – pełne ścieżki użytkownika

Testy Jednostkowe

Plik	Opis	Autor
test_split_calculator.py	Algorytmy podziału (equal, exact, percent, shares)	Wojciech Domański
test_settlement_engine.py	Obliczanie sald, minimalizacja przelewów	Wojciech Gosiewski
test_expense_schema.py	Walidacja schematów wydatków	Wojciech Gosiewski
test_scheduler.py	Obliczanie next_run (weekly, monthly)	Jakub Gerwel
test_security.py	Haszowanie bcrypt, JWT (create, decode, expired)	Dominik Gąsowski
test_group_crud.py	CRUD grup, członkowie, role	Arkadiusz Gnidziejko
test_user.py	CRUD użytkowników	Dominik Gąsowski
test_schemas.py	Walidacja schematów Pydantic	Arkadiusz Gnidziejko

Zaawansowana Automatyzacja

Testy API (test_api.py)

18 testów API bez przeglądarki (Playwright APIRequestContext):

Klasa	Testy	Autor	Opis
-------	-------	-------	------

Klasa	Testy	Autor	Opis
TestAuthAPI	6	Gosiewski (2), Domański (2), Gerwel (1), Gąsowski (1)	Rejestracja, logowanie, /me, duplikat email, brak tokena
TestGroupAPI	3	Gerwel (1), Domański (1), Gnidziejko (1)	Tworzenie grupy, dodawanie członka, brak dostępu (403)
TestExpenseAPI	3	Gerwel (2), Gosiewski (1)	Tworzenie wydatku, pobieranie po ID, usuwanie
TestAccountAPI	3	Gnidziejko (3)	Tworzenie konta, listowanie, transakcja
TestSettlementAPI	3	Gąsowski (2), Gosiewski (1)	Settlement summary, settle up, brak autoryzacji (401)

Testy z mockowaniem danych (test_mocked.py)

15 testów z `page.route()` – mockowanie odpowiedzi API:

Klasa	Testy	Autor	Co mockujemy
TestMockGroupsDashboard	3	Gosiewski (1), Domański (1), Gerwel (1)	/auth/me, /groups – lista grup, stan pusty, nazwa użytkownika
TestMockExpenseList	3	Domański (1), Gnidziejko (1), Gąsowski (1)	Szczegóły grupy, wydatki, salda – renderowanie listy
TestMockAccounts	3	Gosiewski (1), Gerwel (1), Gnidziejko (1)	/accounts – konta, stan pusty, Total Balance
TestMockCurrencyAPI	3	Domański (1), Gnidziejko (1), Gąsowski (1)	/currency/rate – kurs walut (NBP), błąd 500, wolna odpowiedź
TestMockMembersTab	3	Gosiewski (1), Gerwel (1), Gąsowski (1)	Szczegóły grupy – zakładka Members, role, nagłówek

Testy z zapisem stanu uwierzytelnienia (test_storage_state.py)

3 testy zespołowe z użyciem Playwright `storageState`:

Test	Autor	Opis
TC-SS-001	Wojciech Domański	Dostęp do dashboardu bez logowania (storageState zawiera JWT)
TC-SS-002	Arkadiusz Gnidziejko	Tworzenie grupy bez ponownego logowania

Test	Autor	Opis
TC-SS-003	Jakub Gerwel	Nawigacja do strony Balances bez przekierowania na login

Mechanizm: fixture `authenticated_state` loguje się raz przez UI, zapisuje stan przeglądarki (localStorage + cookies) do pliku JSON za pomocą `context.storage_state(path=...)`. Kolejne testy tworzą nowy kontekst z `browser.new_context(storage_state=path)` – są od razu zalogowane.

Informacje techniczne

Uruchomienie aplikacji

```
docker compose up --build -d
```

Aplikacja: <http://localhost:3000> | API: <http://localhost:8000> | Swagger: <http://localhost:8000/docs>

Instalacja zależności testowych

```
pip install -r backend/requirements.txt  
playwright install chromium
```

Uruchomienie poszczególnych testów

Wszystkie testy (z raportem HTML):

```
pytest tests/ --html=reports/test_report.html --self-contained-html
```

Tylko testy jednostkowe (nie wymagają uruchomionej aplikacji):

```
pytest tests/unit/
```

Tylko testy E2E frontendowe (wymagają uruchomionej aplikacji na localhost:3000):

```
pytest tests/e2e/test_accounts_balances.py \  
       tests/e2e/test_expenses.py \
```

```
tests/e2e/test_groups.py \  
tests/e2e/test_recurring_expenses.py \  
tests/e2e/dg/ --headed
```

Tylko testy API (wymagają uruchomionej aplikacji na localhost:3000):

```
pytest tests/e2e/test_api.py
```

Tylko testy z mockowaniem `page.route()` (wymagają uruchomionego frontendu na localhost:3000, backend NIE jest wymagany):

```
pytest tests/e2e/test_mocked.py --headed
```

Tylko testy `storageState` (wymagają uruchomionej aplikacji na localhost:3000):

```
pytest tests/e2e/test_storage_state.py --headed
```

Uruchomienie testów w trybie headless (bez okna przeglądarki):

```
pytest tests/e2e/
```

Uruchomienie testów z widoczną przeglądarką:

```
pytest tests/e2e/ --headed
```

Generowanie raportu

```
pytest tests/ --html=reports/test_report.html --self-contained-html --jur
```

Raporty:

- `reports/test_report.html` – raport HTML (do otwarcia w przeglądarce)
- `reports/junit_report.xml` – raport JUnit XML

Wersje oprogramowania

Narzędzie	Wersja
-----------	--------

Narzędzie	Wersja
Python	3.13
pytest	9.x
Playwright	1.x(latest)
Node.js	20+
PostgreSQL	16
Docker	27+